



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BENGALURU – 560100, KARNATAKA, INDIA**

B.TECH. (CSE)

V SEMESTER

UE24CS341A – SOFTWARE ENGINEERING

SOFTWARE REQUIREMENTS SPECIFICATION

ON

**UpSkill-AI: An AI-Powered Upskilling Recommendation and Automation
System for Students**

SUBMITTED BY

Name	SRN
VANSH SHARMA	PES2UG24AM179
SHUBHIKA PRADEEP	PES2UG24AM157
HEMANTH KUMAR R	PES2UG24AM202
YAJAT SONI	PES2UG24AM185

Revision History

Name	Date	Reason For Changes	Version
VANSH SHARMA, SHUBHIKA PRADEEP, HEMANTH KUMAR R, YAJAT SONI	6 th September 2026	Initial draft	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for UpSkill-AI, version 1.0, an upskilling recommendation and progress tracking system for students.

This SRS covers the complete system, including all four subsystems:

- Student Profile and Skill Extraction
- Skill Gap Analysis and Personalized Roadmap
- Progress Tracking and Adaptive Adjustment
- Resource Catalogue, Authentication and Role-Based Access

It defines what the system must do. It does not define how the system will be built. Design decisions are covered in the separate Architecture and Design documents.

1.2 Intended Audience and Reading Suggestions

This document is written for the following readers.

Reader	What to read	Why
Development team	All sections	Full specification of what must be built
Faculty evaluator	Sections 1, 2, 5, 6	Scope, features, and non-functional requirements
Testers	Sections 5, 6, Appendix C	Functional requirements and traceability to test cases
Project manager	Sections 1, 2, 5	Scope and feature breakdown for planning
New reader	Sections 1, 2, then Appendix A	Overview first, then terminology

Readers unfamiliar with the domain should read Appendix A before Section 5, because Section 5 uses terms such as skill vector, skill gap, and canonical skill with precise meanings.

1.3 Product Scope

What the product is. UpSkill-AI is a web application that tells a student what to learn next, and in what order, to reach a chosen target role.

The problem it solves. Students know they need to upskill but do not know what to learn. Generic course platforms recommend by popularity, not by the individual student's existing skills or their target role. Students end up taking courses that repeat what they already know, or courses they are not ready for because they lack the prerequisites.

How it works. The system builds a profile of the skills a student already has, drawn from their courses, projects, and resume. It compares that profile against the skills required for a target role. The difference is the student's skill gap. The system then recommends specific learning resources from a curated catalogue that close the highest-priority gaps, ordered so that prerequisites come first. As the student completes items, their profile updates and the recommendations change.

Objectives:

- Reduce the time a student spends deciding what to learn.
- Ensure recommended learning is ordered correctly, so a student is never recommended an advanced item before its prerequisite.
- Give faculty mentors visibility into student progress so they can intervene early.
- Produce recommendations that are explainable, so that for every recommendation the student can see which skill gap it closes.

Out of scope for version 1.0:

- Hosting or delivering learning content. The system links to external resources; it does not host them.
- Payment or enrolment with third-party course providers.
- Job placement or application submission.
- Native mobile applications. The system is web-based and responsive.
- Automatic verification of external certificates against the issuing authority.

1.4 References

Ref	Title	Version	Source
R1	Jackfruit Software Engineering Mini-Project Guidelines 2026, UE24CS341A	2026	Course handout
R2	IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications	1998	IEEE
R3	UpSkill-AI Project Proposal and Synopsis	1.0	Team submission
R4	UpSkill-AI High Level Architecture Document	1.0	To be produced
R5	UpSkill-AI Validation Specification	1.0	To be produced
R6	YouTube Data API v3 Documentation	v3	Google Developers

2. Overall Description

2.1 Product Perspective

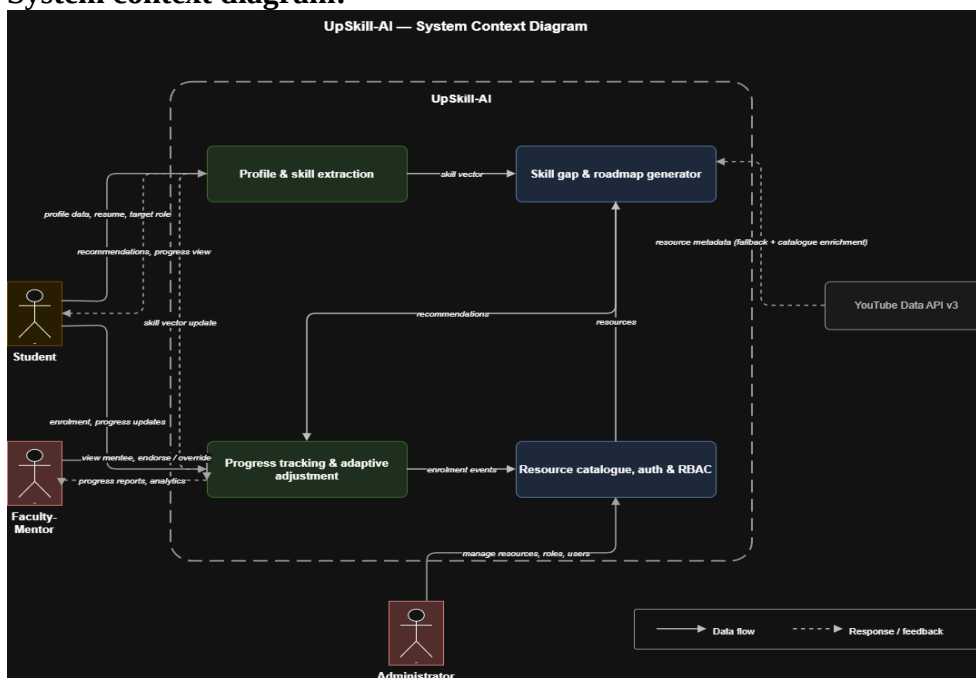
UpSkill-AI is a new, self-contained product. It is not a replacement for an existing system and not a member of an existing product family.

It sits between two things that already exist but are not connected:

- The student's academic and project record, which records what they have already learned.
- The body of available learning resources, consisting of courses, certifications, and project ideas.

Neither side knows about the other. UpSkill-AI is the layer that connects them by translating both into a common language of canonical skills.

System context diagram:



Major interfaces:

- Skill taxonomy — the shared catalogue of canonical skills, their levels, and their prerequisite relationships. Every subsystem reads from it.
- Skill vector — the representation of a student's current skills, produced by the profile subsystem and consumed by the gap analysis subsystem.
- Resource catalogue — the system of record for all recommendable learning resources. It is populated by administrators and enriched from external sources such as the YouTube Data API v3.
- External resource links — the system stores metadata and links for resources held outside the system. It does not host them.

2.2 Product Functions

For students:

- Create and maintain a profile of courses, projects, and skills.
- Upload a resume and have skills extracted from it automatically.
- Review, correct, add, and remove extracted skills.
- Select a target role.
- View their skill gap against that role.
- Receive a ranked, ordered list of recommended learning resources.
- See an explanation of why each item was recommended.
- Enrol in, track, and mark completion of recommended items.
- View their progress over time and see which gaps have closed.

For faculty mentors:

- View the profiles and progress of assigned students.
- Endorse or override recommendations for a student.
- View reports on group progress and identify students who have stalled.

For administrators:

- Manage the skill taxonomy by adding skills, setting levels, and defining prerequisites.
- Manage the resource catalogue by adding, tagging, and moderating learning resources.
- Define target roles and their required skill sets.
- Manage users and role assignments.

Grouping of functions:

Group	Functions
Profile and skill extraction	Profile creation and editing, resume upload, skill extraction, skill normalisation, skill vector computation, target role selection
Skill gap analysis and roadmap	Gap computation, ranking, prerequisite sequencing, roadmap generation, recommendation explanation
Progress tracking	Enrolment, progress states, completion evidence, skill vector update, dashboards, reminders
Catalogue, authentication and access	Resource creation and tagging, catalogue population, search and filter, authentication, role-based access, faculty views, analytics

2.3 User Classes and Characteristics

The system has three human user classes and one automated internal process.

- Student (primary user). Uses the system most frequently. Logs in, maintains a profile of courses, projects, and skills, selects a target role, views the skill gap analysis and

recommended roadmap, enrolls in learning resources, and tracks progress over time. Assumed to have basic computer literacy but no technical expertise. This is the most important class to satisfy, since all core functional requirements exist to serve it.

- Faculty-Mentor (secondary user). Uses the system less frequently. Views the profiles, progress, and recommendation status of assigned students, and may endorse or override a recommendation for an assigned student. Cannot modify a student's profile and cannot perform catalogue or user management. Assumed to have moderate technical familiarity.
- Administrator (secondary user). Maintains the skill taxonomy, the target roles and their required-skill mappings, and the resource catalogue. Manages user accounts and role assignments. Assumed to have moderate to high technical familiarity.
- Recommendation engine (automated internal process, not a human user). Re-evaluates a student's skill vector and progress data to regenerate or re-rank the roadmap. It is triggered by system events rather than by a human user.

2.4 Operating Environment

- Frontend: Next.js (React) web application, server-rendered and responsive, accessible via any modern web browser (Chrome, Firefox, Edge — latest two major versions) on desktop or mobile viewport widths.
- Backend: Python with the FastAPI framework, providing a REST API layer handling authentication, skill-gap computation, recommendation logic, and catalogue management.
- Database: PostgreSQL relational database storing user accounts, student profiles, the skill taxonomy, role-skill mappings, the resource catalogue, progress records, and completion evidence metadata.
- Authentication: Token-based sessions using JSON Web Tokens (JWT); locally managed passwords stored using Argon2 or bcrypt.
- External dependency: YouTube Data API v3 (free tier), queried to populate and enrich the resource catalogue and available as a live fallback when the catalogue has no resource for a skill.
- Hosting: Deployable on any standard cloud virtual machine or platform-as-a-service supporting Python and Node.js runtimes. The exact provider is a team infrastructure decision.
- Client requirement: Internet connectivity is required for all functionality.

2.5 Design and Implementation Constraints

- The technology stack is fixed by team decision: Next.js (React) for the frontend, Python with FastAPI for the backend, and PostgreSQL for persistent storage. All modules must be implemented within this stack for consistency across team members' features.
- Recommendation logic in Phase 1 is a rule-based, level-aware skill-gap matcher rather than a trained machine-learning model. This keeps the feature buildable within the semester while satisfying the AI-powered framing, and is explicitly designed to be extensible to a learned model later.
- The resource catalogue is the system of record for recommendable resources. The YouTube Data API v3 is used to populate and enrich the catalogue and as a live fallback; it is not queried on every recommendation request. Because the free tier has a daily query quota, external results must be cached and reused per skill.
- The system must comply with the course-mandated engineering process: version control on GitHub, pull-request-based code review, containerisation with Docker, continuous integration and delivery with Jenkins, unit testing with pytest, and static analysis with SonarQube.
- Student data must be stored securely and be accessible only to the owning student and to authorised faculty and administrator accounts, with role-based access enforced on the server.

2.6 Assumptions and Dependencies

- Assumes students will input reasonably accurate and up-to-date profile information; recommendation quality depends directly on this.
- Assumes a maintained internal mapping of target roles to required skills and levels exists and is kept current by administrators.
- Depends on the continued availability and free-tier quota of the YouTube Data API v3. If quota is exhausted or the API terms change, catalogue population falls back to manually curated entries.
- Assumes the database schema for skills, roles, resources, and progress is agreed jointly across all four feature owners, since several features read from and write to shared entities.
- Assumes reliable internet connectivity on the client side, consistent with Section 2.4.

3. External Interface Requirements

3.1 User Interfaces

Key screens relevant to the overall product:

- Login / Signup screen — standard email-password form with validation error messages shown inline.
- Student Dashboard — landing screen after login; shows a summary of current skills, the selected target role, and a call to action to view the skill gap analysis.
- Skill Gap and Roadmap screen — displays the computed gap between current and required skills as a list or progress-bar view, with each missing skill expandable to show the recommended learning resources.
- Progress Tracker screen — shows completed versus pending recommended items and progress over time.
- Catalogue screen — lets students search and filter available resources, and lets administrators create and moderate them.
- Faculty-Mentor screen — lists assigned mentees and their progress and recommendation status, with endorse and override controls.

UI conventions to be followed consistently across all screens:

- A persistent top navigation bar with links to Dashboard, Roadmap, Progress, Profile, and Catalogue.
- Inline, plain-language error messages rather than raw error codes, since the primary user class is non-technical.
- Loading indicators for any screen that depends on a live external call.

Detailed UI mockups and wireframes are documented separately in a design file and are out of scope for this SRS section.

3.2 Software Interfaces

Software Interface	Purpose	Input	Output
Web frontend ↔ Backend	Transfers user actions and data requests	Profile edits, enrolment, progress updates, completion evidence, dashboard requests	Profiles, gap analysis, roadmap, progress, milestones, gaps-closed data
Backend ↔ Database	Stores and retrieves persistent data	User, profile, skill, role, resource, progress, and evidence records	Stored records and query results
Backend ↔ YouTube Data API v3	Populates and enriches the resource catalogue	Skill keyword, query parameters	Candidate resources (title, source, URL) for admin review or fallback display

Progress Tracking ↔ Skill Vector Service	Updates a student's skill state after learning evidence	Completed resource, assessment score, evidence	Updated skill vector and confidence values
Progress Tracking ↔ Recommendation Engine	Triggers re-ranking after relevant progress changes	Updated skill vector, completion state, target-role context	Re-ranked recommendations and updated roadmap
Progress Tracking ↔ Notification Service	Sends reminders for stalled learning paths	Progress state, stall condition, reminder rules	Reminder or notification status

3.2.1 Frontend–Backend Interface

The web frontend shall communicate with the backend through REST APIs. The frontend shall submit profile edits, enrolment actions, progress updates, completion status, completion evidence metadata, self-assessment quiz results, and dashboard requests. The backend shall return updated profiles, gap analysis, roadmaps, progress, milestone information, gaps-closed information, and the status of any adaptive recommendation update.

3.2.2 Database Interface

The backend shall communicate with the PostgreSQL database to store and retrieve user accounts, profiles, the skill taxonomy, role–skill mappings, catalogue resources, enrolment records, learning progress, completion status, assessment scores, completion evidence metadata, milestones, streaks, and history.

3.2.3 Skill Vector Interface

When a student completes a learning resource or provides valid performance evidence, the progress tracking module shall provide the relevant learning outcome and evidence information to the skill vector service. The skill vector service shall return the student's updated skill state and confidence values for the affected skills.

3.2.4 Recommendation Engine Interface

The progress tracking module shall notify the recommendation engine when a student's skill vector or learning state changes in a way that affects the current roadmap. The recommendation engine shall use the updated student state to re-rank applicable recommendations and return the revised roadmap.

3.2.5 Notification Service Interface

The progress tracking module shall provide stalled-path and reminder events to the notification service. The notification service shall deliver reminders to students according to the configured notification rules.

3.3 Communications Interfaces

- Client–server communication shall use HTTPS.
- Backend services shall expose REST APIs.
- API request and response bodies shall use JSON as the primary data-exchange format.
- Authentication and session credentials shall be transmitted securely.
- Sensitive student information and completion evidence metadata shall not be transmitted over unsecured HTTP connections.
- Standard HTTP response codes shall be used for success and error conditions.

3.4 Hardware Interfaces

No dedicated hardware interface is required. Students, faculty mentors, and administrators access the application using standard computers, smartphones, or tablets through a supported web browser. The system does not depend on specialised hardware.

4. Analysis Models

4.1 Use Case Diagram

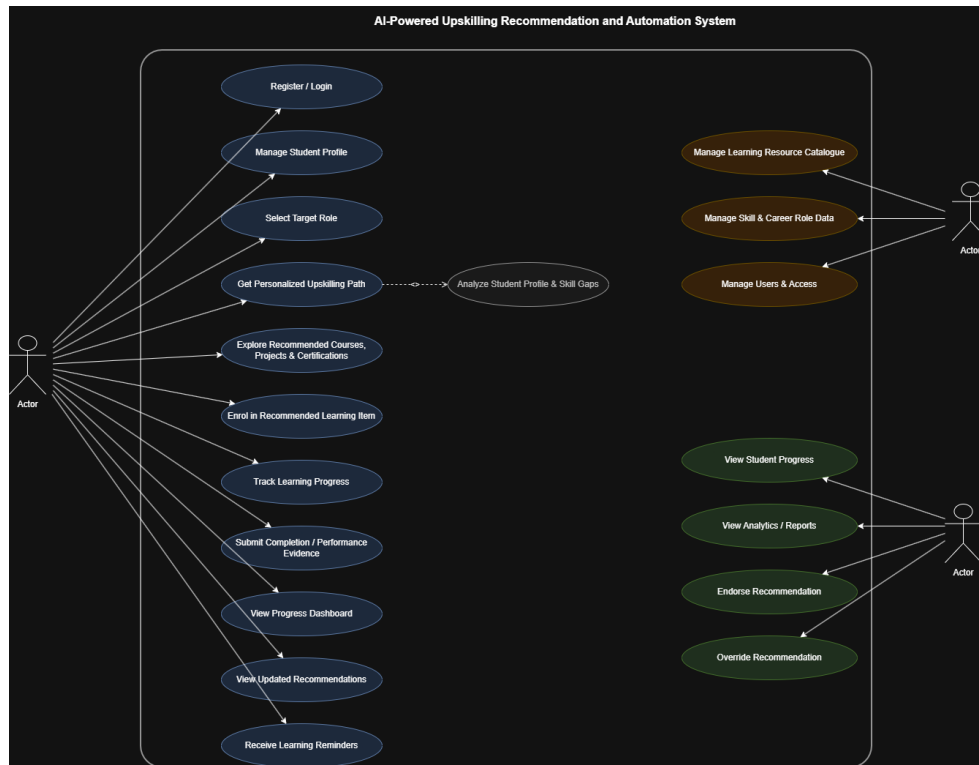


Figure 1 — Use Case Diagram for UpSkill-AI

The system has three actors: Student, Administrator, and Faculty-Mentor.

- Student use cases: Register / Login, Manage Student Profile, Select Target Role, Get Personalized Upskilling Path (includes Analyze Student Profile and Skill Gaps), Explore Recommended Courses, Projects and Certifications, Enrol in Recommended Learning Resource, Track Learning Progress, Submit Completion / Performance Evidence, View Progress Dashboard, View Updated Recommendations, and Receive Learning Reminders.
- Administrator use cases: Manage Learning Resource Catalogue, Manage Skill and Target Role Data, and Manage Users and Access.
- Faculty-Mentor use cases: View Student Progress, View Analytics and Reports, Endorse Recommendation, and Override Recommendation.

The progress workflow shows that completion or performance evidence updates the student's skill vector, which in turn triggers re-ranking of recommendations.

4.2 Entity Relationship Diagram

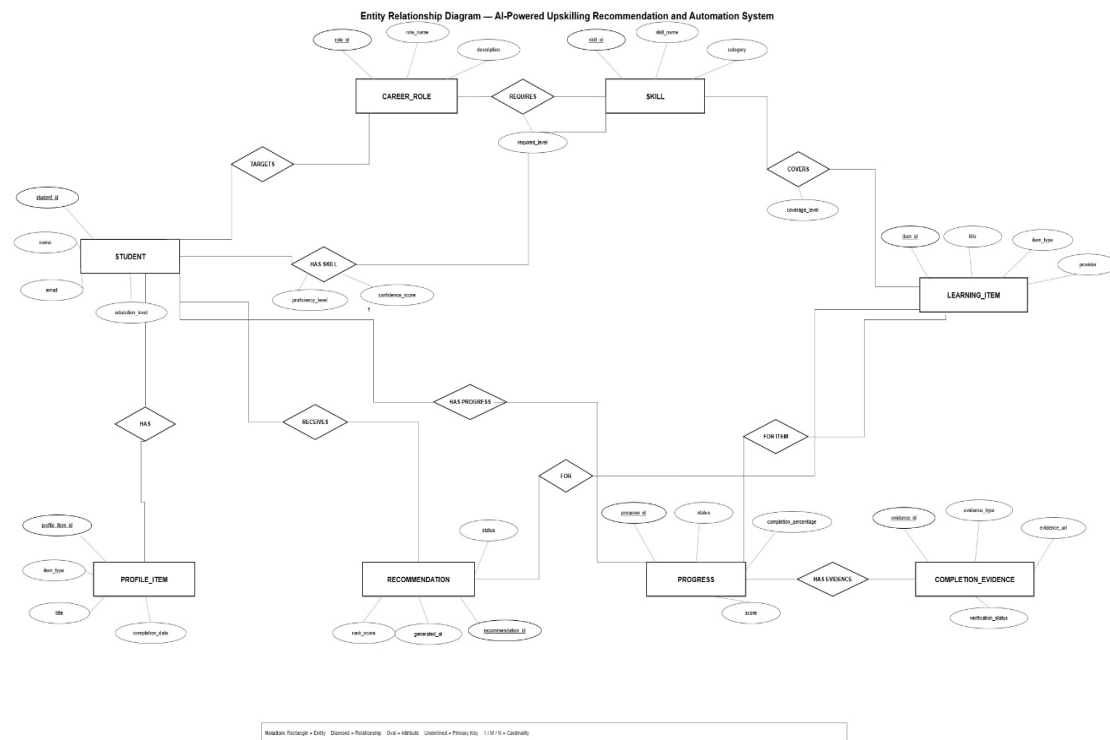


Figure 2 — Entity Relationship Diagram for UpSkill-AI

The core domain entities and their relationships are:

- **STUDENT** (student_id, name, email, education_level) — **TARGETS** one **CAREER_ROLE**; **HAS** many **PROFILE_ITEMS**; **HAS_SKILL** relates **STUDENT** to **SKILL** with proficiency_level and confidence_score; **RECEIVES** many **RECOMMENDATIONS**; **HAS_PROGRESS** relates **STUDENT** to **LEARNING_ITEM**.
- **CAREER_ROLE** (role_id, role_name, description) — **REQUIRES** many **SKILLS** with a required_level.
- **SKILL** (skill_id, skill_name, category) — **COVERS** relates **SKILL** to **LEARNING_ITEM** with a coverage_level.
- **LEARNING_ITEM** (item_id, title, item_type, provider) — the catalogue resource entity.
- **PROFILE_ITEM** (profile_item_id, item_type, title, completion_date) — a course or project in a student's profile.
- **RECOMMENDATION** (recommendation_id, rank_score, generated_at, status).
- **PROGRESS** (progress_id, student_id, resource_id, status, completion_percentage, score).
- **COMPLETION_EVIDENCE** (evidence_id, progress_id, evidence_type, evidence_url, submitted_date, verification_status).

Relationships include **STUDENT 1:N PROGRESS**, **LEARNING_ITEM 1:N PROGRESS**, and **PROGRESS 1:N COMPLETION_EVIDENCE**. Access control, audit, and notification entities are maintained by the corresponding subsystems and will be reflected in the final version of Figure 2.

5. System Features

5.1 Student Profile and Skill Extraction

5.1.1 Description and Priority

This feature builds and maintains the record of what a student already knows. A student enters their academic courses, projects, and self-declared skills, and may upload a resume. The system reads this

input, identifies the skills mentioned, and maps every skill it finds to a single canonical skill in the taxonomy. It then computes a skill vector — the structured summary of the student's current skills and the level the student holds each one at.

This feature is the entry point to the system. No other feature can function without it. The skill gap analysis feature consumes the skill vector as its primary input, and the progress tracking feature updates it. The demanding part of this feature is skill normalisation: students write skills inconsistently, and if normalisation fails, the gap analysis and every recommendation that follows from it are wrong.

Priority: High.

Rating	Score (1 to 9)	Reason
Benefit	9	No part of the system works without a skill vector
Penalty	9	If absent, no recommendation can be generated at all
Cost	6	Resume parsing and skill normalisation require substantial effort
Risk	6	Extraction accuracy on unstructured resumes is the main uncertainty

5.1.2 Stimulus/Response Sequences

Sequence 1: Student creates a profile manually

Step	Actor	Action or response
1	Student	Logs in and opens the profile page for the first time
2	System	Displays an empty profile form with sections for courses, projects, and skills
3	Student	Enters a course name, grade, and completion term, then submits
4	System	Validates the entry, saves it, and displays it in the course list
5	System	Identifies skills implied by the course, resolves them to canonical skills, and adds them to the pending skills list for review
6	Student	Reviews the pending skills and confirms, edits, or removes each one
7	System	Recomputes the skill vector and confirms that the profile is updated

Sequence 2: Student uploads a resume

Step	Actor	Action or response
1	Student	Selects a resume file in PDF or DOCX format and uploads it
2	System	Validates the file type and size, and rejects the file with a clear message if either is invalid
3	System	Displays a processing indicator
4	System	Extracts text, identifies candidate skills, and resolves each to a canonical skill
5	System	Displays the extracted skills, each with its confidence and the source sentence in which it was found
6	Student	Confirms correct extractions, corrects wrong ones, removes false positives, and adds any skills that were missed
7	System	Saves the confirmed skills and recomputes the skill vector
8	System	If no skills were found, informs the student and offers manual entry instead

Sequence 3: Student selects a target role

Step	Actor	Action or response
1	Student	Opens the target role selector
2	System	Displays the available target roles with a short description of each

3	Student	Selects a role
4	System	Saves the selection and records that the profile is ready for gap analysis
5	System	Displays a summary of skills held, skills required, and the number of gaps

Sequence 4: Student corrects an incorrectly extracted skill

Step	Actor	Action or response
1	Student	Opens the skill list and selects a skill to edit
2	System	Displays the skill, its current level, and its source
3	Student	Changes the level or removes the skill
4	System	Saves the change, recomputes the skill vector, and marks existing recommendations as stale

Sequence 5: Skill vector is updated after a completed resource

Step	Actor	Action or response
1	Progress tracking feature	Reports that a student has completed a learning resource
2	System	Reads the skills that the resource teaches and the level it teaches them at
3	System	Raises the student's level for those skills where the new level is higher than the level currently held
4	System	Recomputes and stores the updated skill vector
5	System	Records the change in the profile history together with its source

5.1.3 Functional Requirements

Profile management

REQ-F1-01: The system shall allow an authenticated student to create exactly one profile linked to their user account.

REQ-F1-02: The system shall allow a student to add, edit, and delete course entries. Each course entry shall contain a course name, an optional course code, a grade, and a completion term.

REQ-F1-03: The system shall allow a student to add, edit, and delete project entries. Each project entry shall contain a title, a description of up to 2000 characters, and an optional repository URL.

REQ-F1-04: The system shall allow a student to manually add a skill by selecting a canonical skill from the taxonomy and declaring a level of Beginner, Intermediate, or Advanced.

REQ-F1-05: The system shall reject a manually added skill that is not present in the skill taxonomy, and shall display the closest matching canonical skills as suggestions.

REQ-F1-06: The system shall record, for every skill in a student's profile, whether its source was manual entry, a course, a resume, or a completed resource.

REQ-F1-07: The system shall maintain a history of all changes to a student's skill set, recording the skill, the previous level, the new level, the source, and the time of the change.

Resume upload and extraction

REQ-F1-08: The system shall accept resume uploads in PDF and DOCX formats only, and shall reject all other file types with the message "Only PDF and DOCX files are accepted."

REQ-F1-09: The system shall reject any uploaded file larger than 5 MB with the message "File exceeds the 5 MB limit."

REQ-F1-10: The system shall extract plain text from an accepted resume file.

REQ-F1-11: If text extraction yields fewer than 100 characters, the system shall inform the student that the resume could not be read, and shall offer manual skill entry as an alternative.

REQ-F1-12: The system shall identify candidate skill mentions in the extracted text.

REQ-F1-13: The system shall record, for each extracted skill, a confidence measure that the skill is genuinely held by the student, and shall support ordering and filtering of extracted skills by that measure.

REQ-F1-14: The system shall display each extracted skill together with its confidence and the source sentence in which it was found.

REQ-F1-15: The system shall not add any resume-extracted skill to the profile until the student has explicitly confirmed it.

REQ-F1-16: The system shall allow a student to confirm, edit the level of, or reject each extracted skill individually.

REQ-F1-17: The system shall allow a student to confirm, in a single action, all extracted skills whose confidence is at or above a configurable high-confidence threshold.

REQ-F1-18: The system shall retain an uploaded resume file for no longer than 30 days after processing, and shall delete it thereafter.

Skill normalisation

REQ-F1-19: The system shall resolve every extracted or entered skill string to exactly one canonical skill in the taxonomy.

REQ-F1-20: The system shall recognise known variant forms of a skill name and resolve them to the same canonical skill. For example, "ReactJS", "React.js", and "react" shall all resolve to the canonical skill "React".

REQ-F1-21: The system shall ignore differences of letter case and surrounding whitespace when resolving a skill string.

REQ-F1-22: If a skill string cannot be resolved to any canonical skill, the system shall record it as unresolved and present it to the student for manual selection of the correct canonical skill. It shall not discard the string silently.

REQ-F1-23: The system shall record every unresolved skill string and shall make these records available to administrators for review.

REQ-F1-24: The system shall not create duplicate skill entries. Where a skill is derived from more than one source, the system shall retain a single entry at the highest level found and shall record all contributing sources.

Skill vector computation

REQ-F1-25: The system shall compute a skill vector for a student, consisting of the set of canonical skills the student holds and the level held for each.

REQ-F1-26: The system shall recompute the skill vector whenever any skill in the profile is added, edited, or removed.

REQ-F1-27: The system shall make the current skill vector available to the skill gap analysis feature in the skill vector format defined in Appendix B.

REQ-F1-28: Where a skill is held at more than one level from different sources, the system shall use the highest of those levels.

REQ-F1-29: The system shall accept skill vector updates arising from completed learning resources, and shall raise a skill's level only where the incoming level is higher than the level currently held.

REQ-F1-30: The system shall mark existing recommendations as stale whenever the skill vector changes.

Target role selection

REQ-F1-31: The system shall present the available target roles, each with a name and a short description.

REQ-F1-32: The system shall allow a student to hold exactly one active target role at a time.

REQ-F1-33: The system shall allow a student to change their target role, and shall mark existing recommendations as stale when this occurs.

REQ-F1-34: The system shall display, once a target role is selected, a summary showing the number of required skills, the number of skills already held, and the number of gaps.

Validation and error handling

REQ-F1-35: The system shall reject a profile save in which a required field is empty, and shall identify each invalid field to the student.

REQ-F1-36: The system shall reject any grade value outside the valid range defined in Appendix B.

REQ-F1-37: The system shall reject a project entry whose description exceeds 2000 characters, and shall display the current character count.

REQ-F1-38: If resume processing fails for any reason, the system shall display a clear error message and shall leave the existing profile unchanged.

REQ-F1-39: The system shall ensure that a student can view and modify only their own profile.

REQ-F1-40: The system shall permit a faculty mentor to view, but not modify, the profile of a student assigned to them.

5.2 Skill Gap Analysis and Personalized Roadmap Generator

5.2.1 Description and Priority

This feature compares a student's current skill vector against the skill requirements of their selected target role. It computes the missing skills — the gap — where a skill is either not held or held below the required level, ranks them by relevance to the target role, and generates a personalized roadmap consisting of ranked learning resources drawn from the catalogue, ordered so that prerequisites come first. For every recommended resource, the feature records which skill gap it closes.

Priority: High — this is the core value proposition of the application and is a direct prerequisite for the Progress Tracking feature (5.3).

5.2.2 Stimulus/Response Sequences

- Student selects or updates a target role → System retrieves the required-skills-and-levels set for that role.
- System compares required skills against the student's skill vector → System computes and displays the level-aware skill gap.
- System ranks the gap and selects catalogue resources that cover the missing skills → System generates the ordered roadmap.
- Student opens a missing skill → System displays the recommended resources with title, source, link, and the gap each one closes.
- Skill vector changes via profile edit or completed resource → System is triggered to recompute the gap and re-rank the roadmap.
- Error case: If the catalogue has no resource for a missing skill → System queries the YouTube Data API v3 or shows a curated fallback for that skill, without interrupting the student's flow.

5.2.3 Functional Requirements

REQ-F2-01: The system shall retrieve the required-skills-and-levels set associated with the student's selected target role.

REQ-F2-02: The system shall compute the skill gap as the set of skills that are required by the target role but are either not held in the student's skill vector or held below the required level.

REQ-F2-03: The system shall rank the skills in the gap by their relevance and importance to the target role.

REQ-F2-04: The system shall display the computed skill gap to the student in a readable, prioritized list.

REQ-F2-05: For each skill in the gap, the system shall present ranked learning resources drawn from the resource catalogue that cover that skill.

REQ-F2-06: The system shall order the generated roadmap so that a resource covering a prerequisite skill appears before a resource that depends on it.

REQ-F2-07: For each recommended resource, the system shall record and display the skill gap that the resource closes.

REQ-F2-08: The system shall recompute the skill gap and roadmap automatically whenever the student's skill vector or target role changes.

REQ-F2-09: If the catalogue contains no resource covering a missing skill, the system shall query the YouTube Data API v3, or display a locally stored fallback list, rather than showing an error to the student.

REQ-F2-10: The system shall cache external resource-lookup results per skill for a configurable period to avoid redundant external API calls.

REQ-F2-11: The system shall record the date and time of the last skill-gap recalculation for each student for use by the Progress Tracking feature.

5.3 Progress Tracking and Adaptive Adjustment

5.3.1 Description and Priority

The Progress Tracking and Adaptive Adjustment feature enables students to enrol in recommended learning resources, record learning progress, submit evidence of completion, and monitor their progress toward a target role. The system records completion states and performance evidence such as certificate uploads and self-assessment quiz scores. When a learning resource is completed and valid evidence is recorded, the system updates the student's skill vector and triggers re-ranking of applicable recommendations. The feature also provides progress dashboards, milestones, and reminders for stalled learning paths.

Priority: High.

5.3.2 Stimulus/Response Sequences

ID	Stimulus	System Response
SR-F3-01	Student enrolls in a recommended learning resource	System creates an enrolment/progress record with an initial progress state
SR-F3-02	Student updates progress on a learning resource	System validates the update and stores the new progress state or completion percentage
SR-F3-03	Student marks a learning resource as completed	System records the completion event and requests valid completion evidence when required
SR-F3-04	Student uploads completion evidence	System stores the evidence metadata and associates it with the corresponding progress record
SR-F3-05	Student submits a self-assessment quiz score	System validates and records the score as performance evidence
SR-F3-06	Valid completion or performance evidence is recorded	System updates the student's skill vector for the affected skills

SR-F3-07	Student skill vector is updated	System triggers recommendation re-ranking using the updated student state
SR-F3-08	Student opens the progress dashboard	System displays progress over time, gaps closed, streaks, and milestones
SR-F3-09	Learning path meets the stalled-progress condition	System identifies the stalled condition and schedules or sends a reminder
SR-F3-10	Recommendation set changes after a progress update	System displays the updated roadmap or recommendation status to the student

5.3.3 Functional Requirements

REQ-F3-01: The system shall allow a student to enrol in a recommended learning resource.

REQ-F3-02: The system shall record the student's progress state for each enrolled learning resource.

REQ-F3-03: The system shall allow the student to update the completion percentage of an enrolled learning resource.

REQ-F3-04: The system shall allow the student to mark an enrolled learning resource as completed.

REQ-F3-05: The system shall support recording completion evidence associated with a learning resource, including certificate metadata or self-assessment quiz results.

REQ-F3-06: The system shall validate required completion evidence before treating an evidence-based completion event as valid.

REQ-F3-07: The system shall record the performance score submitted through a supported self-assessment quiz.

REQ-F3-08: The system shall update the student's skill vector when valid completion or performance evidence is recorded.

REQ-F3-09: The system shall trigger recommendation re-ranking when a student's skill vector is updated by a relevant learning outcome.

REQ-F3-10: The system shall display the student's progress over time for enrolled learning resources.

REQ-F3-11: The system shall display the number or proportion of identified skill gaps that have been closed.

REQ-F3-12: The system shall track and display applicable learning streaks and milestones.

REQ-F3-13: The system shall identify learning paths that meet the configured stalled-progress condition.

REQ-F3-14: The system shall generate a notification or reminder for a stalled learning path according to the configured notification rules.

REQ-F3-15: The system shall preserve the student's progress history when a recommendation is re-ranked or changed.

REQ-F3-16: The system shall prevent a completion event from being recorded against an invalid or non-enrolled learning resource.

REQ-F3-17: The system shall display the updated recommendation set or roadmap returned by the recommendation engine after a relevant progress-driven skill-vector update.

5.4 Resource Catalogue, Authentication and Role-Based Access

5.4.1 Description and Priority

This feature provides the shared resource catalogue and access-control layer of the system. It allows administrators to create, update, review, approve, and remove learning resources; allows students to search and filter available resources; and provides authentication and role-based access for Students,

Faculty-Mentors, and Administrators. Faculty-Mentors can view mentee progress and may endorse or override recommendations when appropriate.

Priority: High. The feature is required for secure access to the platform and for maintaining a usable, moderated catalogue of learning resources.

5.4.2 Stimulus/Response Sequences

Stimulus	Action	System Response
Catalogue search	User enters a keyword or applies filters	System returns matching resources that the user is authorized to view
Resource creation	Administrator submits a new resource	System validates required fields and stores the resource; if moderation is configured, the resource remains pending until approved
Resource moderation	Administrator reviews a pending resource	System records the decision and makes approved resources available to eligible users
Student authentication	Student submits valid login credentials	System authenticates the user and grants access to student-authorized functionality
Unauthorized access	User attempts to access an endpoint outside their role permissions	System denies the request and records the security event where applicable
Faculty mentee view	Faculty-Mentor selects an assigned mentee	System displays the mentee's permitted progress, skill-gap, and recommendation information
Faculty override or endorsement	Faculty-Mentor endorses or overrides a recommendation	System records the action, identifies the responsible faculty user, and reflects the decision in the student's recommendation view

5.4.3 Functional Requirements

REQ-F4-01: The system shall support authentication for Students, Faculty-Mentors, and Administrators.

REQ-F4-02: The system shall assign each authenticated user a role that determines access to protected functionality.

REQ-F4-03: The system shall prevent Students from accessing Faculty-Mentor or Administrator-only endpoints.

REQ-F4-04: The system shall prevent Faculty-Mentors from performing Administrator-only catalogue management operations unless explicitly authorized.

REQ-F4-05: The system shall allow Administrators to create, edit, deactivate, and delete catalogue resources.

REQ-F4-06: The system shall support the resource types course, certification, and project idea.

REQ-F4-07: Each catalogue resource shall store, at minimum, a title, description, resource type, skill tags, level, estimated duration, and source URL.

REQ-F4-08: The system shall validate required catalogue fields before accepting a resource.

REQ-F4-09: The system shall support catalogue search using resource title, description, and skill tags.

REQ-F4-10: The system shall allow users to filter resources by relevant attributes such as skill, level, type, and duration.

REQ-F4-11: The system shall support an administrative moderation state for catalogue resources.

REQ-F4-12: The system shall make only approved or otherwise explicitly published resources available to Students.

REQ-F4-13: The system shall allow Faculty-Mentors to view the progress and recommendation status of their assigned mentees.

REQ-F4-14: The system shall allow Faculty-Mentors to endorse a recommendation for an assigned mentee.

REQ-F4-15: The system shall allow authorized Faculty-Mentors to override a recommendation and record the reason for the override.

REQ-F4-16: The system shall maintain an audit record for administrative catalogue changes and faculty recommendation overrides and endorsements.

REQ-F4-17: The system shall expose analytics and reports appropriate to the user's role.

REQ-F4-18: The system shall return an authorization error when a user attempts to access a protected operation without sufficient privileges.

5.4.4 Feature Constraints and Validation

- A resource cannot be published when required metadata is missing or invalid.
- Skill tags used by catalogue resources shall correspond to canonical skills in the system taxonomy.
- Source URLs shall be stored as valid URLs and shall be checked for basic format validity.
- Role permissions shall be enforced server-side; hiding a UI element alone shall not constitute access control.
- Faculty-Mentor access shall be limited to mentees assigned to that faculty user.
- Administrative actions and recommendation overrides shall be attributable to an authenticated user.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Requirement
NFR-PERF-01	For normal catalogue searches, the system should return results within 2 seconds for at least 95% of requests under the expected project workload.
NFR-PERF-02	Authenticated API requests should normally receive a response within 2 seconds, excluding long-running file processing or external service calls.
NFR-PERF-03	Recommendation and gap-analysis requests should complete within 5 seconds for a typical student profile and target role.
NFR-PERF-04	The system shall paginate large catalogue result sets rather than returning an unbounded collection in a single response.
NFR-PERF-05	The system should support concurrent access by the expected academic cohort without noticeable degradation of core functions.

6.2 Safety Requirements

ID	Requirement
NFR-SAFE-01	The system shall not present recommendation output as a guaranteed career outcome; recommendations shall be presented as guidance based on available profile and catalogue data.
NFR-SAFE-02	The system shall preserve user-submitted evidence files and profile data according to the application's retention and access rules.
NFR-SAFE-03	The system shall handle invalid, missing, or unexpectedly formatted uploaded files without crashing the application.
NFR-SAFE-04	The system shall fail safely when an external service used for parsing, recommendations, or notifications is unavailable.

6.3 Security Requirements

ID	Requirement
----	-------------

NFR-SEC-01	The system shall require authentication before accessing protected student, faculty, or administrative data.
NFR-SEC-02	The system shall enforce role-based authorization on the server for every protected operation.
NFR-SEC-03	Passwords, if locally managed, shall never be stored in plaintext and shall use a secure password-hashing mechanism.
NFR-SEC-04	Authentication and session credentials shall be transmitted and stored using appropriate secure mechanisms.
NFR-SEC-05	The system shall validate and sanitize user input to reduce risks such as injection, malicious file upload, and malformed requests.
NFR-SEC-06	Uploaded files shall be checked for permitted file types and reasonable size limits before processing or storage.
NFR-SEC-07	A student shall only be able to access their own private profile, evidence, and progress data.
NFR-SEC-08	A Faculty-Mentor shall only be able to access student information for assigned mentees.
NFR-SEC-09	Administrative and faculty actions affecting catalogue data or recommendations shall be auditable.
NFR-SEC-10	Sensitive application errors shall not expose passwords, tokens, internal stack traces, or other confidential implementation details to end users.

6.4 Software Quality Attributes

ID	Requirement
NFR-QUAL-01	Usability — The interface shall present recommendations, skill gaps, and resource metadata in a clear and understandable manner.
NFR-QUAL-02	Maintainability — Major modules such as profile management, recommendation logic, progress tracking, and catalogue management shall remain logically separated.
NFR-QUAL-03	Testability — Core recommendation, authorization, catalogue validation, and progress-update logic shall be implementable as isolated automated tests.
NFR-QUAL-04	Reliability — A failure in a non-critical service such as notification delivery shall not prevent core profile, catalogue, or progress operations from functioning.
NFR-QUAL-05	Scalability — The catalogue and user data model shall support growth in resources and users without requiring a redesign of core entities.
NFR-QUAL-06	Accessibility — User-facing pages should provide readable text, meaningful labels, keyboard-accessible controls, and sufficient semantic structure.
NFR-QUAL-07	Interoperability — Resource source URLs and standard file formats shall be handled using commonly supported web and document formats.
NFR-QUAL-08	Consistency — Canonical skill identifiers shall be used across profiles, resources, roles, and recommendation outputs.

6.5 Business Rules

ID	Requirement
BR-01	Every catalogue resource must be associated with at least one canonical skill tag before publication.
BR-02	A resource's difficulty level shall be selected from the levels supported by the system taxonomy.
BR-03	Only authenticated users may access personalized recommendations or progress information.
BR-04	Students may view and enrol in resources available to their role but may not modify the canonical catalogue.
BR-05	Faculty-Mentors may act only on students assigned to them.
BR-06	Administrators have authority over catalogue moderation and system-level catalogue changes.
BR-07	A recommendation override shall not silently delete the underlying algorithmic recommendation; the original recommendation and the override action shall remain auditable.

BR-08	A resource marked inactive shall not appear in new student recommendations or normal catalogue search results unless explicitly requested by an authorized administrator.
BR-09	Analytics shown to a user shall be limited to data that the user's role permits them to access.

7. Other Requirements

7.1 Data Retention and Privacy

- Uploaded resume files shall be retained for no longer than 30 days after processing and then deleted, consistent with REQ-F1-18.
- Completion evidence and profile data shall be retained for the duration of the student's active account and handled according to the access rules in Section 6.3.
- A student shall be able to request deletion of their own profile data; on deletion, personally identifying data shall be removed while anonymised aggregate analytics may be retained.

7.2 Internationalization

- Version 1.0 supports the English language only for the user interface and content.
- Dates and times shall be displayed in the local time zone of the accessing client where the browser provides it.

7.3 Legal and Licensing

- Use of the YouTube Data API v3 shall comply with the API's terms of service and quota limits.
- The system shall store links and metadata for external learning resources and shall not host or redistribute third-party copyrighted content.
- Where a resource is displayed, its original source shall be attributed.

7.4 Deployment and Installation

- The application shall be containerised with Docker so that the frontend, backend, and database can be provisioned in a reproducible environment.
- Builds, tests, and deployments shall run through the Jenkins continuous integration and delivery pipeline.
- Static analysis via SonarQube shall be part of the pipeline, and a build that fails the agreed quality gate shall not be promoted.

7.5 Future Extensibility

- The rule-based recommendation logic shall be replaceable by a trained machine-learning model without redesigning the profile, catalogue, or progress subsystems.
- The data model shall permit the addition of further user roles beyond the current three.

Appendix A: Glossary

Term	Definition
Administrator	A user role that maintains the skill taxonomy, target roles, resource catalogue, and user accounts.
Audit record	A stored, attributable record of an administrative or faculty action, such as a catalogue change or a recommendation override.
Canonical skill	The single agreed name for a skill in the taxonomy. All variant forms resolve to it, so that one skill is never counted twice under different names.
Career role	See Target role. Stored as the CAREER_ROLE entity in the data model.

Confidence	A measure of how certain the system is that an extracted skill is genuinely held by the student.
Completion evidence	Proof that a student finished a recommended resource, such as an uploaded certificate or a passed self-assessment.
Endorsement	A faculty-mentor's confirmation of a recommendation for an assigned student.
Faculty-Mentor	A user role that can view assigned students' profiles and progress and endorse or override recommendations, but cannot modify a student's profile or manage the catalogue.
Learning resource / Learning item	An external learning item — a course, certification, or project idea — held in the catalogue with its skill tags, level, and duration. Stored as the LEARNING_ITEM entity.
Level	The depth at which a skill is held or taught. One of Beginner, Intermediate, or Advanced.
Milestone	A recorded progress achievement shown on the student dashboard.
Moderation	The administrative review that determines whether a submitted resource becomes available to students.
Override	A faculty-mentor's replacement of an algorithmic recommendation, recorded with a reason and kept auditable.
Prerequisite	A skill that must be held before another skill can reasonably be learned.
Profile	The complete record of a student's courses, projects, skills, and target role.
RBAC	Role-Based Access Control. Permissions granted according to a user's role rather than individually.
Roadmap	The ordered sequence of recommended resources, arranged so that prerequisites come before the skills that depend on them.
Skill gap	A skill required by the target role that the student either does not hold, or holds at a lower level than required.
Skill taxonomy	The structured catalogue of all canonical skills, their levels, and the prerequisite relationships between them.
Skill vector	The structured representation of the skills a student currently holds and the level held for each.
Stale recommendation	A recommendation generated before a change to the skill vector or the target role, and therefore possibly no longer valid.
Stalled path	A learning path on which a student has made no qualifying progress within the configured period.
Streak	A count of consecutive periods in which a student made qualifying learning progress.
Target role	The role a student is working towards, defined as a required set of skills and levels. Stored as the CAREER_ROLE entity.
Unresolved skill	A skill string that could not be matched to any canonical skill, and is held for manual review rather than discarded.

Appendix B: Field Layouts

Levels used throughout are the ordered set {Beginner, Intermediate, Advanced}. Grades used in course entries are letter grades drawn from the set {S, A, B, C, D, E, F}, or an equivalent percentage in the range 0–100; any value outside these is invalid (see REQ-F1-36).

Skill Vector Format (referenced by REQ-F1-27)

Field	Data Type	Description	Mandatory
canonical_skill_id	Integer	Identifier of the canonical skill in the taxonomy	Y
canonical_skill_name	String (60)	Display name of the canonical skill	Y
level	Enum	One of Beginner, Intermediate, Advanced	Y
confidence	Decimal (0–1)	Confidence that the skill is genuinely held	N
source	Enum	One of manual, course, resume, completion	Y

Profile and Skill Fields

Field	Length	Data Type	Description	Mandatory
student_id	10	Numeric	Unique identifier for the student	Y
name	60	String	Student name	Y
email	120	String	Student email, used for login	Y
education_level	30	String	Current education level	N
course_name	100	String	Name of a completed course	Y
course_code	15	String	Optional course code	N
grade	3	String / Numeric	Letter grade {S,A,B,C,D,E,F} or 0–100	Y
completion_term	10	String	Term in which the course was completed	Y
project_title	100	String	Title of a project	Y
project_description	2000	String	Project description	Y
repository_url	255	String (URL)	Optional project repository link	N

Target Role and Gap Fields

Field	Length	Data Type	Description	Mandatory
target_role	60	String	Target role selected by the student	Y
required_skills	list	Alphanumeric list	Skills and required levels for the target role	Y
skill_gap	list	Alphanumeric list	Missing or under-level skills = required – held	N (system-generated)
last_updated	8	Date	Date the skill gap was last recalculated	N (system-generated)

Resource Catalogue Fields

Field	Length	Data Type	Description	Mandatory
item_id	10	Numeric	Unique identifier for the learning resource	Y
title	150	String	Resource title	Y
description	1000	String	Resource description	Y
item_type	15	Enum	course, certification, or project idea	Y
skill_tags	list	Alphanumeric list	Canonical skills the resource covers	Y
level	15	Enum	Beginner, Intermediate, or Advanced	Y
duration	10	String	Estimated duration	N
source_url	255	String (URL)	Link to the external resource	Y
source	30	String	Resource source, e.g. YouTube, freeCodeCamp	N
moderation_state	15	Enum	pending, approved, inactive	Y

Progress and Evidence Fields

Field	Length	Data Type	Description	Mandatory
progress_id	10	Numeric	Unique identifier for a progress record	Y
resource_id	10	Numeric	Learning resource being tracked	Y
status	15	Enum	enrolled, in-progress, completed	Y
completion_percentage	3	Numeric	Percentage complete, 0–100	N

score	3	Numeric	Self-assessment score, 0–100	N
evidence_id	10	Numeric	Unique identifier for completion evidence	Y
evidence_type	20	Enum	certificate or quiz	Y
evidence_url	255	String (URL)	Link to uploaded evidence	N
submitted_date	8	Date	Date the evidence was submitted	N
verification_status	15	Enum	pending, verified, rejected	N

User and Access Fields

Field	Length	Data Type	Description	Mandatory
user_id	10	Numeric	Unique identifier for a user account	Y
role	15	Enum	Student, Faculty-Mentor, Administrator	Y
password_hash	255	String	Securely hashed password (never stored in plaintext)	Y
mentor_id	10	Numeric	Faculty-mentor assigned to a student	N

Appendix C: Requirement Traceability Matrix

Architecture Reference abbreviations: HLD = High Level Architecture Document (to be produced). Design Reference abbreviations: SDD = Software Design Document (to be produced). Code File Reference values are indicative placeholders to be updated when module names are finalised. Test Case IDs follow the convention TC-Fn-nn (unit/integration) and STC-Fn-nn (system test).

Feature 1 — Student Profile and Skill Extraction

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-F1-01	One profile per student account	HLD §Profile	SDD §Profile	profile/models.py	TC-F1-01	STC-F1-01
2	REQ-F1-02	Add/edit/delete course entries	HLD §Profile	SDD §Profile	profile/views.py	TC-F1-02	STC-F1-02
3	REQ-F1-03	Add/edit/delete project entries	HLD §Profile	SDD §Profile	profile/views.py	TC-F1-03	STC-F1-03
4	REQ-F1-04	Manual skill add from taxonomy	HLD §Profile	SDD §Skills	skills/views.py	TC-F1-04	STC-F1-04
5	REQ-F1-05	Reject non-taxonomy skill	HLD §Profile	SDD §Skills	skills/validators.py	TC-F1-05	STC-F1-05
6	REQ-F1-06	Record skill source	HLD §Profile	SDD §Skills	skills/models.py	TC-F1-06	STC-F1-06
7	REQ-F1-07	Skill change history	HLD §Profile	SDD §Skills	skills/history.py	TC-F1-07	STC-F1-07
8	REQ-F1-08	Accept PDF/DOCX only	HLD §Profile	SDD §Resume	resume/validators.py	TC-F1-08	STC-F1-08
9	REQ-F1-09	Reject file >5 MB	HLD §Profile	SDD §Resume	resume/validators.py	TC-F1-09	STC-F1-09
10	REQ-F1-10	Extract resume text	HLD §Profile	SDD §Resume	resume/parser.py	TC-F1-10	STC-F1-10
11	REQ-F1-11	Handle unreadable resume	HLD §Profile	SDD §Resume	resume/parser.py	TC-F1-11	STC-F1-11
12	REQ-F1-12	Identify candidate skills in text	HLD §Profile	SDD §Resume	resume/extractor.py	TC-F1-12	STC-F1-12
13	REQ-F1-13	Record extraction confidence	HLD §Profile	SDD §Resume	resume/extractor.py	TC-F1-13	STC-F1-13
14	REQ-F1-14	Display skill + confidence + sentence	HLD §Profile	SDD §Resume	resume/views.py	TC-F1-14	STC-F1-14
15	REQ-F1-15	Require confirmation before add	HLD §Profile	SDD §Resume	resume/views.py	TC-F1-15	STC-F1-15

16	REQ-F1-16	Confirm/edit/reject per skill	HLD §Profile	SDD §Resume	resume/views.py	TC-F1-16	STC-F1-16
17	REQ-F1-17	Bulk-confirm high-confidence skills	HLD §Profile	SDD §Resume	resume/views.py	TC-F1-17	STC-F1-17
18	REQ-F1-18	Delete resume after 30 days	HLD §Data	SDD §Data	tasks/cleanup.py	TC-F1-18	STC-F1-18
19	REQ-F1-19	Resolve to one canonical skill	HLD §Profile	SDD §Taxonomy	taxonomy/resolver.py	TC-F1-19	STC-F1-19
20	REQ-F1-20	Recognise skill variant forms	HLD §Profile	SDD §Taxonomy	taxonomy/aliases.py	TC-F1-20	STC-F1-20
21	REQ-F1-21	Case/whitespace-insensitive match	HLD §Profile	SDD §Taxonomy	taxonomy/resolver.py	TC-F1-21	STC-F1-21
22	REQ-F1-22	Hold unresolved skills for review	HLD §Profile	SDD §Taxonomy	taxonomy/resolver.py	TC-F1-22	STC-F1-22
23	REQ-F1-23	Expose unresolved to administrator	HLD §Profile	SDD §Taxonomy	taxonomy/admin.py	TC-F1-23	STC-F1-23
24	REQ-F1-24	No duplicate skill entries	HLD §Profile	SDD §Skills	skills/models.py	TC-F1-24	STC-F1-24
25	REQ-F1-25	Compute skill vector	HLD §Profile	SDD §Skills	skills/vector.py	TC-F1-25	STC-F1-25
26	REQ-F1-26	Recompute on profile change	HLD §Profile	SDD §Skills	skills/signals.py	TC-F1-26	STC-F1-26
27	REQ-F1-27	Expose vector to gap analysis	HLD §Integration	SDD §API	skills/api.py	TC-F1-27	STC-F1-27
28	REQ-F1-28	Use highest level on conflict	HLD §Profile	SDD §Skills	skills/vector.py	TC-F1-28	STC-F1-28
29	REQ-F1-29	Accept vector updates from progress	HLD §Integration	SDD §API	skills/api.py	TC-F1-29	STC-F1-29
30	REQ-F1-30	Mark recommendations stale	HLD §Integration	SDD §Recommendations	recommendations/stale.py	TC-F1-30	STC-F1-30
31	REQ-F1-31	Present target roles	HLD §Profile	SDD §Roles	roles/views.py	TC-F1-31	STC-F1-31
32	REQ-F1-32	One active target role	HLD §Profile	SDD §Roles	roles/models.py	TC-F1-32	STC-F1-32
33	REQ-F1-33	Change target role	HLD §Profile	SDD §Roles	roles/views.py	TC-F1-33	STC-F1-33
34	REQ-F1-34	Show gap summary on role select	HLD §Profile	SDD §Gap	gap/views.py	TC-F1-34	STC-F1-34
35	REQ-F1-35	Reject empty required field	HLD §Profile	SDD §Profile	profile/validators.py	TC-F1-35	STC-F1-35
36	REQ-F1-36	Reject invalid grade value	HLD §Profile	SDD §Profile	profile/validators.py	TC-F1-36	STC-F1-36
37	REQ-F1-37	Reject over-length description	HLD §Profile	SDD §Profile	profile/validators.py	TC-F1-37	STC-F1-37
38	REQ-F1-38	Fail safely on parse error	HLD §Profile	SDD §Resume	resume/parser.py	TC-F1-38	STC-F1-38
39	REQ-F1-39	Student sees only own profile	HLD §Auth	SDD §Auth	auth/permissions.py	TC-F1-39	STC-F1-39
40	REQ-F1-40	Faculty read-only on mentee profile	HLD §Auth	SDD §Auth	auth/permissions.py	TC-F1-40	STC-F1-40

Feature 2 — Skill Gap Analysis and Roadmap

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
41	REQ-F2-01	Retrieve required skills for role	HLD §Gap	SDD §Gap	gap/engine.py	TC-F2-01	STC-F2-01
42	REQ-F2-02	Compute level-aware skill gap	HLD §Gap	SDD §Gap	gap/engine.py	TC-F2-02	STC-F2-02
43	REQ-F2-03	Rank gap by role relevance	HLD §Gap	SDD §Gap	gap/ranker.py	TC-F2-03	STC-F2-03

44	REQ-F2-04	Display prioritized gap list	HLD §Gap	SDD §Gap	gap/views.py	TC-F2-04	STC-F2-04
45	REQ-F2-05	Recommend catalogue resources per skill	HLD §Recommendations	SDD §Recommendations	recommendations/engine.py	TC-F2-05	STC-F2-05
46	REQ-F2-06	Order roadmap by prerequisite	HLD §Recommendations	SDD §Recommendations	recommendations/sequencer.py	TC-F2-06	STC-F2-06
47	REQ-F2-07	Show gap each resource closes	HLD §Recommendations	SDD §Recommendations	recommendations/views.py	TC-F2-07	STC-F2-07
48	REQ-F2-08	Recompute on vector or role change	HLD §Integration	SDD §Gap	gap/signals.py	TC-F2-08	STC-F2-08
49	REQ-F2-09	Fallback to YouTube or curated list	HLD §Integration	SDD §External	external/youtube.py	TC-F2-09	STC-F2-09
50	REQ-F2-10	Cache external resource lookups	HLD §Integration	SDD §External	external/cache.py	TC-F2-10	STC-F2-10
51	REQ-F2-11	Record last recalculation timestamp	HLD §Gap	SDD §Gap	gap/models.py	TC-F2-11	STC-F2-11

Feature 3 — Progress Tracking and Adaptive Adjustment

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
52	REQ-F3-01	Enrol in a learning resource	HLD §Progress	SDD §Progress	progress/views.py	TC-F3-01	STC-F3-01
53	REQ-F3-02	Record progress state	HLD §Progress	SDD §Progress	progress/models.py	TC-F3-02	STC-F3-02
54	REQ-F3-03	Update completion percentage	HLD §Progress	SDD §Progress	progress/views.py	TC-F3-03	STC-F3-03
55	REQ-F3-04	Mark resource completed	HLD §Progress	SDD §Progress	progress/views.py	TC-F3-04	STC-F3-04
56	REQ-F3-05	Record completion evidence	HLD §Progress	SDD §Evidence	evidence/views.py	TC-F3-05	STC-F3-05
57	REQ-F3-06	Validate evidence before completion	HLD §Progress	SDD §Evidence	evidence/validators.py	TC-F3-06	STC-F3-06
58	REQ-F3-07	Record quiz score	HLD §Progress	SDD §Evidence	evidence/views.py	TC-F3-07	STC-F3-07
59	REQ-F3-08	Update skill vector on evidence	HLD §Integration	SDD §Skills	skills/signals.py	TC-F3-08	STC-F3-08
60	REQ-F3-09	Trigger re-ranking on vector update	HLD §Integration	SDD §Recommendations	recommendations/signals.py	TC-F3-09	STC-F3-09
61	REQ-F3-10	Display progress over time	HLD §Progress	SDD §Dashboard	dashboard/views.py	TC-F3-10	STC-F3-10
62	REQ-F3-11	Display gaps closed	HLD §Progress	SDD §Dashboard	dashboard/views.py	TC-F3-11	STC-F3-11
63	REQ-F3-12	Track streaks and milestones	HLD §Progress	SDD §Dashboard	dashboard/milestones.py	TC-F3-12	STC-F3-12
64	REQ-F3-13	Identify stalled learning paths	HLD §Progress	SDD §Notifications	notifications/detector.py	TC-F3-13	STC-F3-13
65	REQ-F3-14	Generate stalled-path reminder	HLD §Progress	SDD §Notifications	notifications/sender.py	TC-F3-14	STC-F3-14
66	REQ-F3-15	Preserve history on re-rank	HLD §Progress	SDD §Progress	progress/models.py	TC-F3-15	STC-F3-15
67	REQ-F3-16	Block completion on non-enrolled item	HLD §Progress	SDD §Progress	progress/validators.py	TC-F3-16	STC-F3-16

68	REQ-F3-17	Display updated roadmap after re-rank	HLD §Recommendations	SDD §Recommendations	recommendations/views.py	TC-F3-17	STC-F3-17
----	-----------	---------------------------------------	----------------------	----------------------	--------------------------	----------	-----------

Feature 4 — Resource Catalogue, Authentication and Access

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
69	REQ-F4-01	Authenticate three user roles	HLD §Auth	SDD §Auth	auth/views.py	TC-F4-01	STC-F4-01
70	REQ-F4-02	Assign role on authentication	HLD §Auth	SDD §Auth	auth/models.py	TC-F4-02	STC-F4-02
71	REQ-F4-03	Block student from elevated endpoints	HLD §Auth	SDD §Auth	auth/permissions.py	TC-F4-03	STC-F4-03
72	REQ-F4-04	Block faculty from admin operations	HLD §Auth	SDD §Auth	auth/permissions.py	TC-F4-04	STC-F4-04
73	REQ-F4-05	Admin catalogue CRUD	HLD §Catalogue	SDD §Catalogue	catalogue/views.py	TC-F4-05	STC-F4-05
74	REQ-F4-06	Support three resource types	HLD §Catalogue	SDD §Catalogue	catalogue/models.py	TC-F4-06	STC-F4-06
75	REQ-F4-07	Store required resource fields	HLD §Catalogue	SDD §Catalogue	catalogue/models.py	TC-F4-07	STC-F4-07
76	REQ-F4-08	Validate catalogue fields	HLD §Catalogue	SDD §Catalogue	catalogue/validators.py	TC-F4-08	STC-F4-08
77	REQ-F4-09	Search catalogue by title/tags	HLD §Catalogue	SDD §Catalogue	catalogue/search.py	TC-F4-09	STC-F4-09
78	REQ-F4-10	Filter resources by attributes	HLD §Catalogue	SDD §Catalogue	catalogue/filters.py	TC-F4-10	STC-F4-10
79	REQ-F4-11	Administrative moderation state	HLD §Catalogue	SDD §Catalogue	catalogue/moderation.py	TC-F4-11	STC-F4-11
80	REQ-F4-12	Only approved resources to students	HLD §Catalogue	SDD §Catalogue	catalogue/filters.py	TC-F4-12	STC-F4-12
81	REQ-F4-13	Faculty view mentee progress	HLD §Faculty	SDD §Faculty	faculty/views.py	TC-F4-13	STC-F4-13
82	REQ-F4-14	Faculty endorse recommendation	HLD §Faculty	SDD §Faculty	faculty/views.py	TC-F4-14	STC-F4-14
83	REQ-F4-15	Faculty override with reason	HLD §Faculty	SDD §Faculty	faculty/views.py	TC-F4-15	STC-F4-15
84	REQ-F4-16	Audit admin and faculty actions	HLD §Auth	SDD §Audit	audit/models.py	TC-F4-16	STC-F4-16
85	REQ-F4-17	Role-appropriate analytics	HLD §Faculty	SDD §Analytics	analytics/views.py	TC-F4-17	STC-F4-17
86	REQ-F4-18	Authorization error on breach	HLD §Auth	SDD §Auth	auth/permissions.py	TC-F4-18	STC-F4-18